The George Washington University

Data Science Program | Columbian College of Arts & Sciences

Time Series Analysis & Modeling

DATS 6313 | Fall 2026

Homework # 7 — Parameter Estimation

Due: Wednesday, November 11, 2026

This lab has two phases. The main purpose of **Phase I** is to implement the maximum-likelihood estimation algorithm called the **Levenberg–Marquardt (LM)** algorithm using a Python program to estimate the coefficients of an ARMA model. The main objective of **Phase II** is to use the `statsmodels` package to estimate the parameters of the known ARMA models (8 examples).
You are working with synthetic data for this lab with known parameters to verify the accuracy of the developed code. At this moment, you should have already checked the accuracy of your GPAC code to estimate the order of an ARMA model using synthetic data in the previous lab. You will estimate the parameters of the ARMA model in this lab using two different approaches. **Use `np.random.seed(6313)` and display all numbers with 3-digit decimal precision. (100 points total: Phase I = 65, Phase II = 35.)**

Phase I

Data generation Use the same code developed in the previous assignment to generate a synthetic dataset per the following input information:

(a) Number of observations:

(b) Enter AR order:

(c) Enter MA order:

(d) Enter coefficients of AR:

(e) Enter coefficients of MA:

Then generate a white noise normally distributed with zero mean and variance of 1, and generate the $y(t)$ output data for the AR process shown below. (You will use the same code to generate data for the other examples in this lab.)

$$y(t) - 0.5\,y(t-1) = e(t)$$

1- Develop a Python program that implements the LM algorithm explained in class and estimates the coefficients of the following ARMA model. The pseudo-code for the LM algorithm is given in the lecture notes and needs to be implemented for this question. The true coefficient is -0.5 and the estimated parameter should converge to the true coefficient as the number of samples goes to infinity. **[15 pts]**

$$y(t) - 0.5\,y(t-1) = e(t)$$

2- Display the confidence interval for the estimated parameter(s). Are all the coefficients statistically important? Why? Justify your answer. **[3 pts]**

3- Display the estimated covariance matrix of the estimated parameters. **[3 pts]**

4- Display the estimated variance of error. **[3 pts]**

5- Display the poles and zeros (roots of the numerator and roots of the denominator). Are there any zero/pole cancellation(s)? Justify your answer. **[4 pts]**

6- Plot the sum of squared error (SSE) versus the number of iterations. **[2 pts]**

7- Repeat the above steps (1–6) for the following 7 examples (5 points each). **[35 pts]**

```
Example 2 -- ARMA(0,1):   y(t) = e(t) + 0.5e(t-1)
Example 3 -- ARMA(1,1):   y(t) + 0.5y(t-1) = e(t) + 0.25e(t-1)
Example 4 -- ARMA(2,0):   y(t) + 0.5y(t-1) + 0.2y(t-2) = e(t)
Example 5 -- ARMA(2,1):   y(t) + 0.5y(t-1) + 0.2y(t-2) = e(t) - 0.5e(t-1)
Example 6 -- ARMA(1,2):   y(t) + 0.5y(t-1) = e(t) + 0.5e(t-1) - 0.4e(t-2)
Example 7 -- ARMA(0,2):   y(t) = e(t) + 0.5e(t-1) - 0.4e(t-2)
Example 8 -- ARMA(2,2):   y(t) + 0.5y(t-1) + 0.2y(t-2) = e(t) + 0.5e(t-1) -
0.4e(t-2)
```

Phase II

In this phase of the lab, you will work on the same examples as the previous phase. You will estimate the coefficients of the ARMA model but using the **`statsmodels` package in Python**.

```
import statsmodels.api as sm
```

8- Using the `statsmodels` package, simulate the random process $y(t)-0.5\,y(t-1) = e(t)$ with $e(t)$ a normally distributed white noise with mean zero and variance 1. The number of observations, the order of the ARMA model, and the corresponding coefficients are entered in question 1. You may use `arma_process.generate_sample(N)`. **[4 pts]**

9- Using the generated data (y) from the previous question and knowing the orders n_a and n_b, estimate the coefficients of the ARMA model $y(t) - 0.5\,y(t-1) = e(t)$ using the `statsmodels` package. Display the estimated coefficient(s) on the console with 3-digit decimal precision. **[3 pts]**

10- Display the summary of the fit with the highlighted coefficient(s) and the confidence interval. Take a screenshot and place it into your report. Does the result match the result from the LM algorithm? **[7 pts]**

11- Repeat steps 8, 9, and 10 for the following 7 examples (3 points each). **[21 pts]**

```
Example 2 -- ARMA(0,1):   y(t) = e(t) + 0.5e(t-1)
Example 3 -- ARMA(1,1):   y(t) + 0.5y(t-1) = e(t) + 0.25e(t-1)
Example 4 -- ARMA(2,0):   y(t) + 0.5y(t-1) + 0.2y(t-2) = e(t)
Example 5 -- ARMA(2,1):   y(t) + 0.5y(t-1) + 0.2y(t-2) = e(t) - 0.5e(t-1)
Example 6 -- ARMA(1,2):   y(t) + 0.5y(t-1) = e(t) + 0.5e(t-1) - 0.4e(t-2)
Example 7 -- ARMA(0,2):   y(t) = e(t) + 0.5e(t-1) - 0.4e(t-2)
Example 8 -- ARMA(2,2):   y(t) + 0.5y(t-1) + 0.2y(t-2) = e(t) + 0.5e(t-1) -
0.4e(t-2)
```

Submission: Upload the formal solution **report (as a single PDF)** plus the **.py file(s)** through **Blackboard** by the due date. *The report must be a single PDF; a Word document or any other format will incur a 5-point penalty.*

```
                               SARIMAX Results
==========================================================================================
Dep. Variable:                          y   No. Observations:                 5000
Model:                 ARIMA(1, 0, 0)   Log Likelihood               -7068.053
Date:                Mon, 03 Apr 2023   AIC                          14140.107
Time:                        23:11:19   BIC                          14153.141
Sample:                             0   HQIC                         14144.675
                                 - 5000
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.4836      0.012     39.178      0.000       0.459       0.508
sigma2         0.9893      0.020     49.969      0.000       0.951       1.028
===================================================================================
Ljung-Box (L1) (Q):                   0.50   Jarque-Bera (JB):                 0.23
Prob(Q):                              0.48   Prob(JB):                         0.89
Heteroskedasticity (H):               0.98   Skew:                            -0.02
Prob(H) (two-sided):                  0.63   Kurtosis:                         3.00
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
The residual is white
      lb_stat  lb_pvalue
20  18.619911   0.546641
```

Figure 1: *

Example. `statsmodels` SARIMAX summary for the $ARMA(1,0)$ fit.